Georgia Tech

XV6: A SHORT INTRO TO A SMALL OS

ALEXEY TUMANOV

CREATING THE NEXT®

CS3210 h4ck3rZ l33t

Georgia Tech

Rohan

Sukrit

Mayank

Souradeep

Garv

Sheetal

Erin

Eric

Michael

Sofia

CREAT

TODAY'S LECTURE

- Administrativia
  - Intro card → due Fri 1/16 on gradescope (95+ submissions)
    - 95+ submission
  - Diagnostic quiz → due Fri 1/16 on gradescope
    - 70+ submissions
  - **Lab 0** :
    - Released: Fri Jan 16, due Fri, Jan 23
    - That's correct, this is *next week*
  - **Lab 1** :
    - Assigned: Fri Jan 23
    - Due: Fri Feb 6
    - ~14 days for this lab

TODAY'S LECTURE

- MLK Day: Mon, Jan 19,
  - We will have NO class for BOTH sections (A,B)
  - No Class Jan 19-20 in CS3210
  - This will help stay in sync across both sections w.r.t. lecture material
- Waitlists , etc
- Phase II reg deadline: Fri, Jan 16
- Piazza: https://piazza.com/gatech/spring2026/cs3210/home
- Gradescope: https://www.gradescope.com/courses/1222433
  - Also, Canvas → Gradescope (left navigation panel)
  - **Lab** submissions will be managed through **gradescope**
  - Quizzes (Supervised Labs, Exams) → administered through Canvas

SUPERVISED LABS

- Supervised Lab Week 0 Quiz
  - No supervised lab quiz this week
- Supervised Lab Week 1+ Quizzes
  - Due **TUE**, 11:59pm
  - Please note, quizzes due SAME day as the supervised lab
- Supervised lab section A01 :
  - Last name A-L: A02
  - Last name M-Z: A04
- Whitaker access requested (and granted ?)

TODAY'S LECTURE

- xv6's core systems:
  - Bootup process – how do we get to c code?
  - Virtual memory – How do you bootstrap memory in an OS?
    - How does xv6 manage memory?
  - Processes – How do we organize user execution?
  - System calls and traps – How are these related?
    - What are these, how do they work?
  - Filesystem – How do we do persistent data and naming?

Recommended reading:

Xv6 book, Appendix A/B.

Georgia Tech

git clone git@github.gatech.edu:cs3210-spring2026/xv6.git

Where should we start?

Perhaps we should start at system start:

Booting

Georgia Tech

- What is booting?
  - Can we just jump straight into main?
  - What does our CPU have to do to boot?
    - When is booting even considered complete?
  - What are different things initialized and when?
    - Register state
    - CPU state
    - Stack

- Booting Process:

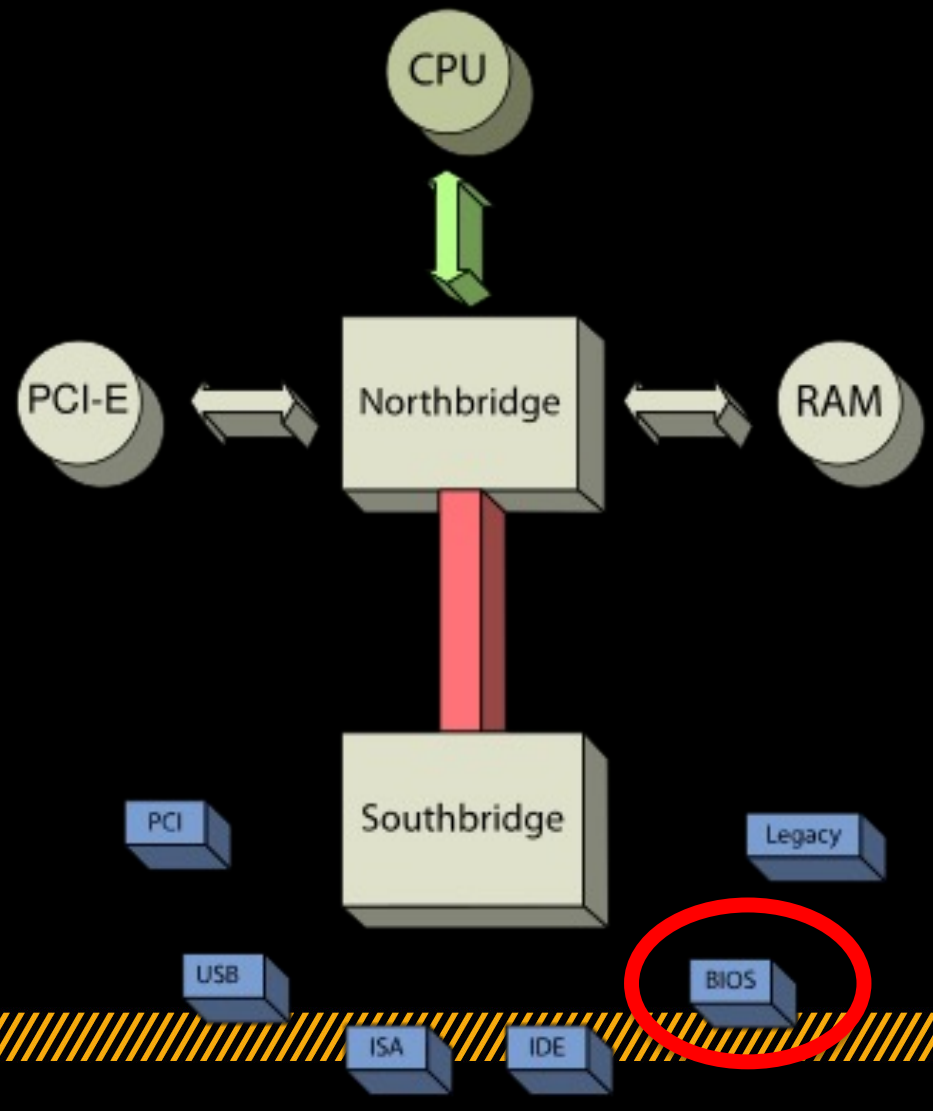
ROM Firmware on the motherboard

First block of the disk (often more)

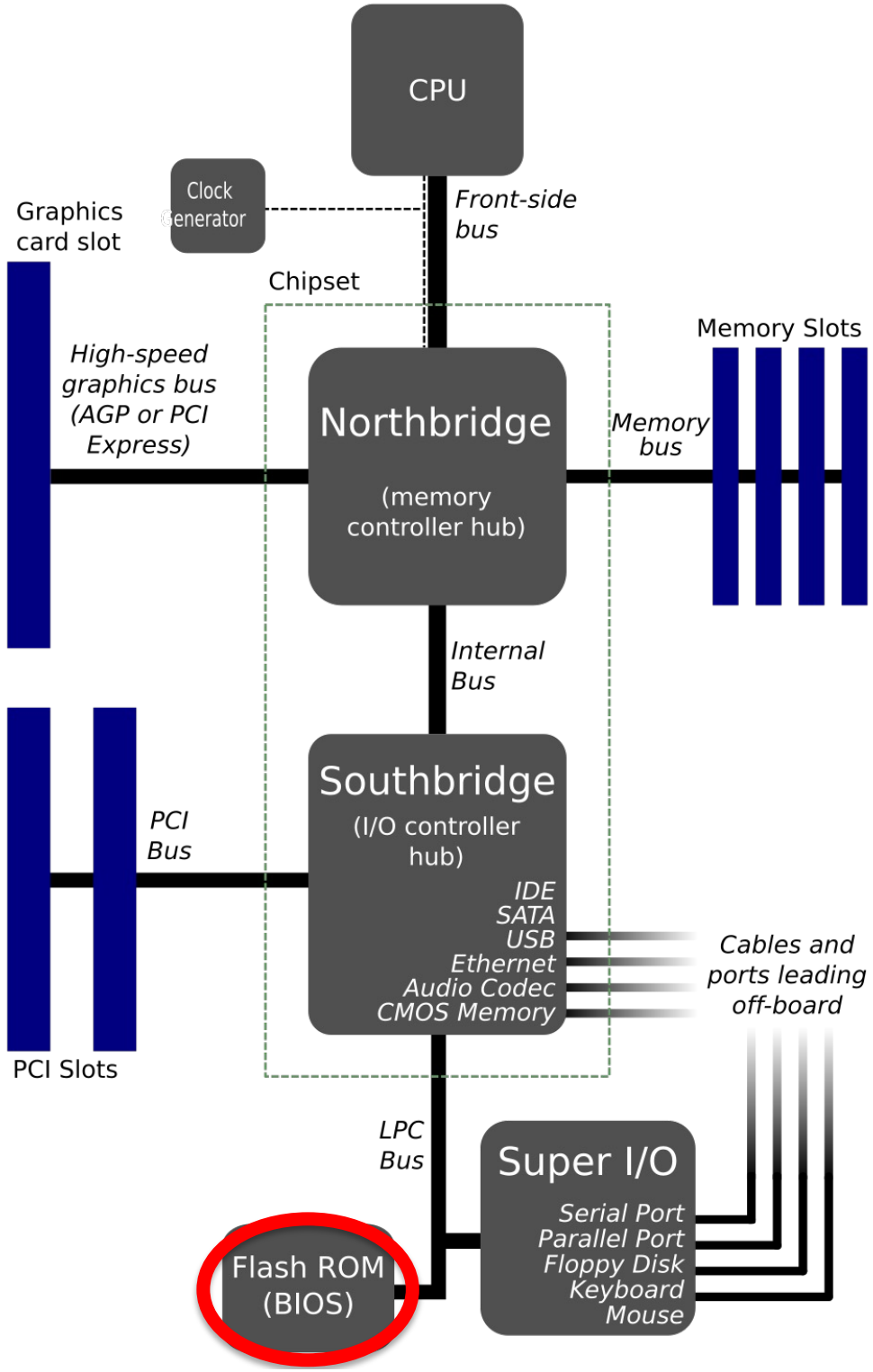
Beginning of kernel partition

CPU – NORTHBRIDGE- BIOS

- Bios location: stored in a dedicated firmware hub or flash chip
- CPU – northbridge – southbridge – BIOS flash chip



Source: Wikipedia



Source: Wikipedia

HIGH LEVEL OVERVIEW

- Booting *Responsibilities* (high level):

Initialize lowest-level hardware details

Load boot block

Initial processor setup

Load kernel & execute

Actual executing kernel

BIOS – BASIC I/O SYSTEM

- Fun Fact : Greek word βίος – means life
  - BIOS brings the computer to life
- Key functionality/responsibility of BIOS:
  - Initialize memory controller
    - Separate chip on the motherboard
    - Manages the flow of data between the CPU and DRAM
    - Multiplexer/demux circuit to interpret physical addresses
    - Instructed which memory lanes to activate
    - Manages the data bus flowing data between the CPU and DRAM
  - Initialize RAM – zero initialize (why?)

BOOTLOADER

- "grub" is an example of a popular bootloader
- Xv6 uses its own (simpler) bootblock found in:
  - Bootasm.S
  - Bootmain.c
- See bootblock directory in repository
  - Bootblock/bootasm.S

```
# Start the first CPU: switch to 32-bit protected mode, jump into C.
# The BIOS loads this code from the first sector of the hard disk into
# memory at physical address 0x7c00 and starts executing in real mode
# with %cs=0 %ip=7c00.

.code16                     # Assemble for 16-bit mode
.globl start
start:
  cli                       # BIOS enabled interrupts; disable
```

WHAT DOES THE XV6 BOOTBLOCK DO?

- 1 line summary: Prepare us to launch the kernel.
- Set up an initial configuration of memory and load the kernel into it.
  - For x86, this means switching from “real-mode” to “protected-mode” (i.e., 32-bit flat memory model).
- Bootasm.S:42 loads a descriptor table and tells the CPU to use 32-bit addressing with 0x0000000 offsets.

```
# Bootstrap GDT
.p2align 2                                # force 4 byte alignment
gdt:
  SEG_NULLASM                             # null seg
  SEG_ASM(STA_X|STA_R, 0x0, 0xffffffff)   # code seg
  SEG_ASM(STA_W, 0x0, 0xffffffff)         # data seg

gdtdesc:
  .word   (gdtdesc - gdt - 1)             # sizeof(gdt) - 1
  .long   gdt                             # address gdt
```

```
lgdt    gdtdesc
movl    %cr0, %eax
orl     $CR0_PE, %eax
movl    %eax, %cr0
```

WHAT DOES THE XV6 BOOTBLOCK DO?

- 1 line summary: Prepare us to launch the kernel.

- Set up an initial configuration of memory and load the kernel into it.
  - For x86, this means switching from “real-mode” to “protected-mode” (i.e., 32-bit flat memory model).
- Bootasm.S:42 loads a descriptor table that tells the CPU to use 32-bit addressing with 0x0000000 offsets.
- Load the kernel code from its ELF binary
- Jumps to kernel’s ELF binary’s entry point.

BOOTLOADER : C

- In bootmain.c, the kernel's ELF headers are checked, parsed and loaded to the correct part of memory (physical address 0x100000).
- Once the kernel is fully loaded from disk into memory, it jumps into the kernel never to return
  - Bootmain.c:47

```
  // Call the entry point from the ELF header.
  // Does not return!
  entry = (void(*)(void))(elf->entry);
  entry();
}
```

- Multiboot loaders are programmed to load different kinds of kernels and so can't get away with this hardcoding.
  - In this class we'll use a simple xv6 single block bootloader

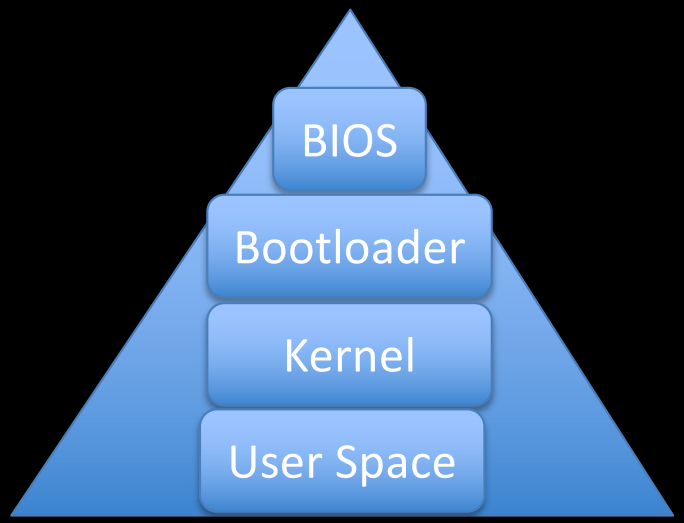
Georgia Tech

- Xv6 bootblock uses a BIOS-loaded master boot record (MBR).
- MBR = sector 0 on the designated boot device, 512 bytes used to bootstrap the OS or multiboot loader.
- Most computers now use UEFI (Unified Extensible Firmware Interface) instead of BIOS and GPT (GUID Partition Table).
  - Allows larger than 2 TiB disks
  - More than 4 partitions
  - More corruption resistant (multiple “master” boot records)
  - Other tricks…

BOOTBLOCK -> KERNEL ENTRY POINT

- Can the kernel jump straight to its main function after the boot block?
  - *Why or why not?*
  - What does the kernel need to do before it can call the c main?

- Kernel's entry point is in kernel/src/asm/entry.S
  - Sets up initial stack
  - Sets up initial page table
  - Jumps to *main()*



- kernel/src/main.c does everything the kernel needs to do to start up and start running programs.

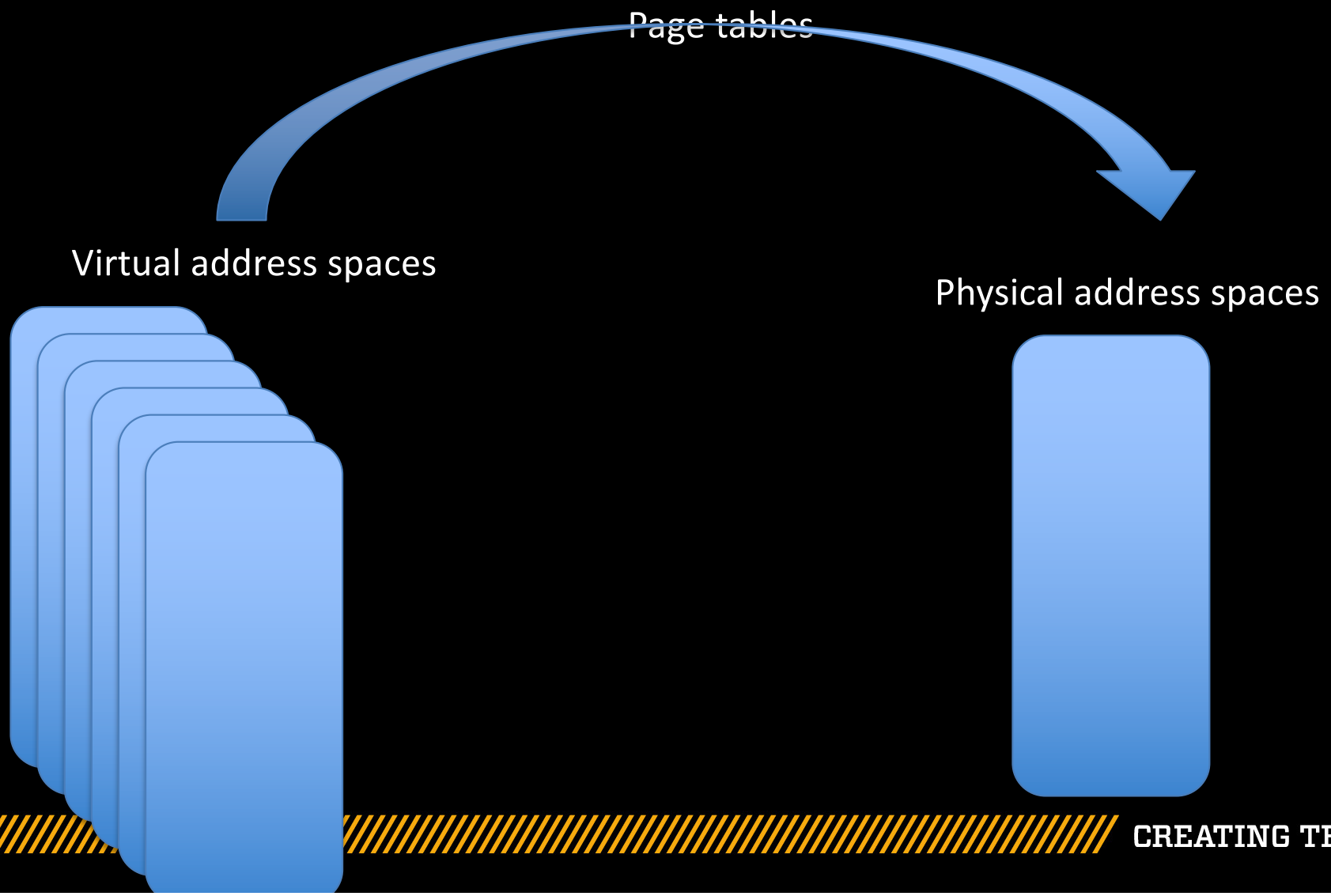
DESIGN OF OPERATING SYSTEMS

- Modern operating systems rarely refer to real, physical memory addresses when addressing code, stack, heap, etc.
- Almost all OS's use virtual memory to create the *abstraction* that every process (and the kernel) has access to all of addressable memory.
  - *Reality*: access to the subset of the physical memory capacity
  - *Mechanism*: paging subsystem
- Later lectures we will cover how this works in great detail.

KERNEL SUBSYSTEMS: VM

Georgia Tech

Virtual Memory



VIRTUAL MEMORY OPERATIONS

- What is a minimum set of operations we need to provide from a VM system?
  - Examples
    - Allocate new page tables
    - Switch between page tables
    - Any others?
- Xv6 implements VM in kernel/src/vm.c:
    - Initialize - inituvm
    - Add/remove memory – allocuvm, deallocuvm
    - Load or switch - switchuvm
    - Destroy - freevm

KERNEL MEMORY SETUP

- How would we access memory at *physical* address 0x100000?
  - XV6 uses a trick – maps all physical address to offset 0x80000000. -- so, can access 0x80100000.
- What about a user process's virtual address 0x7?
  - Evil, but possible (Mickens)
- What about a user process's *virtual* address 0x1008?
  - If the user's page table is mapped, just access it
  - OR Map their page table then access
  - OR Walk the page table, find the physical address, add 0x80000000.

KERNEL SUBSYSTEMS: VM

- Find vm.c
  - The first kernel pagetable is initiated from main.c through kvmalloc
  - Kvmalloc calls switchkvm which sets up a page table for the kernel
  - The page table provides a translation, along with some flags, from virtual to physical addresses,
    - e.g., **VA** 0x80 10 00 00 to **PA** 0x10 00 00 for the kernel.
  - X86-based processors use cr3 (control register 3) as the page-table base pointer.
  - 32-bit OS's use two-level paging; 64-bit OS's use 4-level paging.

VIRTUAL MEMORY SETUP

- Note that every user process has a protected mapping of the kernel page table in its own page table to avoid switching page tables on system calls and interrupts.
- This restricts the amount of memory a process can directly access to the lower 2 GiB of the 4GiB addressable memory.

Processes

KERNEL SUBSYSTEMS: PROCESSES

- Find proc.c (process information)
  - Main.c has more to do to set up the kernel (mostly involving hardware and multicore)
  - The first process happens near the end with userinit.
    - Very first user space process.
  - Sets up the first process called initcode, a piece of assembly linked into the kernel binary.
- Find Initcode.S
  - Bootstraps the first process setting up to use the exec system call and calls init
- user/src/Init.c (user-space program!)
  - Starts the shell after setting up the console input and output.

INITCODE.S

```
# exec(init, argv)
.globl start
start:
  pushl $argv
  pushl $init
  pushl $0  // where caller pc would be
  movl $SYS_exec, %eax
  int $T_SYSCALL

# for(;;) exit();
exit:
  movl $SYS_exit, %eax
  int $T_SYSCALL
  jmp exit

# char init[] = "/init\0";
init:
  .string "/init\0"

# char *argv[] = { init, 0 };
.p2align 2
argv:
  .long init
  .long 0
```

- Back to proc.c
  - Processes are the isolated units that run in xv6
  - Each process runs at a lower privilege than the kernel.
  - Symmetric Multiprocessing provided by functions like
    - Scheduler
    - Sched
    - Swtch (in swtch.S)
    - Yield
  - Interprocess communication provided by
    - Sleep and wakeup

Georgia Tech

System Calls and Traps

- Find vector.S
  - All possible transitions from user to kernel space are here (as well as some kernel to kernel, back to BIOS, errors and failures, etc.)
  - A “trap” occurs on a transition and each has a number called a vector.
- Find trapasm.S
  - This sets up the stack and some registers to transition to kernel.
- Find trap.c
  - Handlers for trap vectors are defined here
  - The most important one is vector 64 (0x40) which is the system call.

- Tvinit sets up the “Interrupt Descriptor Table” (IDT) (more on this in later lectures. For now it basically tells the CPU where to go on traps).
- Each trap has a data structure called a “trap frame” that tells the kernel where the trap is coming from and gives all necessary info to return if desired.
- The trap function (line 37) calls functions to handle all traps including hardware interrupts.

- Find syscall.c
  - Table of “system calls”, calls from user processes to kernel, is set up here
  - Individual system calls have their definitions spread throughout the code base, e.g., pipe creation (sys_pipe) is handled in sysfile.c

Filesystem

Georgia Tech

- Find fs.c
  - Allocates and frees blocks (balloc and bfree) (typically 8 sectors or 4 KiB).
  - Xv6 uses inode blocks (i.e., hard linking) to store file meta-data (links to data and other inodes)
    - ialloc – allocates inodes
    - iupdate – updates (modifies inodes)
    - iget – gets an inodes from cache or disk and caches if needed
    - iput – drops an inode from cache

DESIGN OF OPERATING SYSTEMS

- Find sysfile.c
  - Contains functions for creating files, directories, executing files, and creating pipes (file-like objects)
- Find bio.c
  - Buffer cache caches blocks for efficiency.
  - Read and write blocks
  - Update blocks on disk asynchronously when dirty
- Find log.c
  - File system consistency
  - Logs file system transactions so that they can be recovered in the event of a system crash.

Georgia Tech

- This is just a high level tour. There are many technical details left out.
- All these pieces and others provide the services that a basic OS needs to manage *memory*, run multiple *processes* concurrently, interact with *user space*, handle *events*, and store *data* persistently.
- Xv6 is a "complete" operating system but not advanced.
  - Lacking many important modern features.
- Over the semester we will work to improve it.